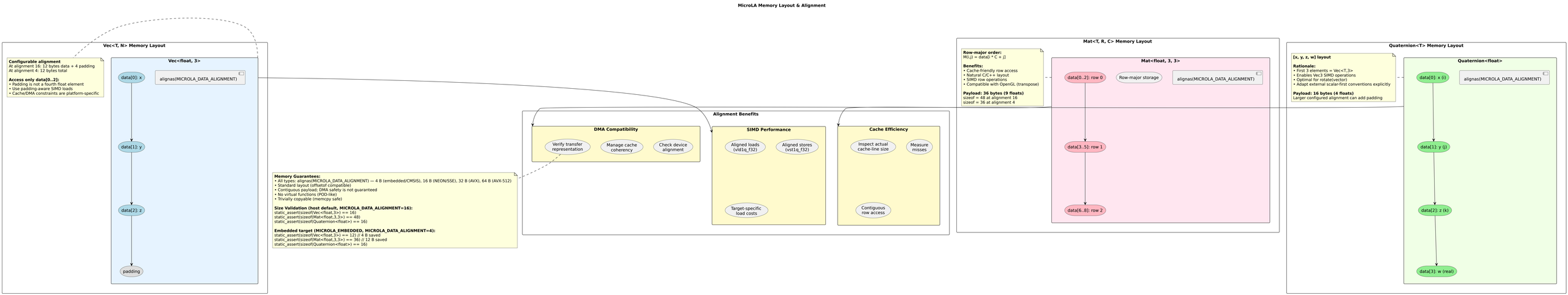
MicroLA Memory Layout & Alignment
Vec<T, N> Memory Layout
Configurable alignment
At alignment 16: 12 bytes data + 4 padding
At alignment 4: 12 bytes total
Access only data[0..2]:
• Padding is not a fourth float element
• Use padding-aware SIMD loads
• Cache/DMA constraints are platform-specific
Vec<float, 3>
data[0]: x
alignas(MICROLA_DATA_ALIGNMENT)
data[1]: y
data[2]: z
padding
Memory Guarantees:
• All types: alignas(MICROLA_DATA_ALIGNMENT) — 4 B (embedded/CMSIS), 16 B (NEON/SSE), 32 B (AVX), 64 B (AVX-512)
• Standard layout (offsetof compatible)
• Contiguous payload; DMA safety is not guaranteed
• No virtual functions (POD-like)
• Trivially copyable (memcpy safe)
Size Validation (host default, MICROLA_DATA_ALIGNMENT=16):
static_assert(sizeof(Vec<float,3>) == 16)
static_assert(sizeof(Mat<float,3,3>) == 48)
static_assert(sizeof(Quaternion<float>) == 16)
Embedded target (MICROLA_EMBEDDED, MICROLA_DATA_ALIGNMENT=4):
static_assert(sizeof(Vec<float,3>) == 12) // 4 B saved
static_assert(sizeof(Mat<float,3,3>) == 36) // 12 B saved
static_assert(sizeof(Quaternion<float>) == 16)
Alignment Benefits
DMA Compatibility
Verify transfer representation
Manage cache coherency
Check device alignment
SIMD Performance
Aligned loads (vld1q_f32)
Aligned stores (vst1q_f32)
Target-specific load costs
Cache Efficiency
Inspect actual cache-line size
Measure misses
Contiguous row access
Mat<T, R, C> Memory Layout
Row-major order:
M(i,j) = data[i * C + j]
Benefits:
• Cache-friendly row access
• Natural C/C++ layout
• SIMD row operations
• Compatible with OpenGL (transpose)
Payload: 36 bytes (9 floats)
sizeof = 48 at alignment 16
sizeof = 36 at alignment 4
Mat<float, 3, 3>
data[0..2]: row 0
Row-major storage
alignas(MICROLA_DATA_ALIGNMENT)
data[3..5]: row 1
data[6..8]: row 2
Quaternion<T> Memory Layout
[x, y, z, w] layout
Rationale:
• First 3 elements = Vec<T,3>
• Enables Vec3 SIMD operations
• Optimal for rotate(vector)
• Adapt external scalar-first conventions explicitly
Payload: 16 bytes (4 floats)
Larger configured alignment can add padding
Quaternion<float>
data[0]: x (i)
alignas(MICROLA_DATA_ALIGNMENT)
data[1]: y (j)
data[2]: z (k)
data[3]: w (real)